University & Department: School of Computing, Queen's University, Kingston, Canada

Course CISC 886 – Cloud Computing

Title Project: End-to-End Cloud-Based Chat Assistant on AWS

Team member:

Name	NetID	ID
Lamya Mamdouh Sayed	25nfnx	20596347
Esraa Mohammed El Sayed	25htmp	20596301
Alaa Ehab Mohamed elshenawy	25fsvf	20596313

Date: April 28, 2026

GitHub Repository: https://github.com/LamyaMamdouh/LlamaChat-OASST1

Section 1 — System Architecture:

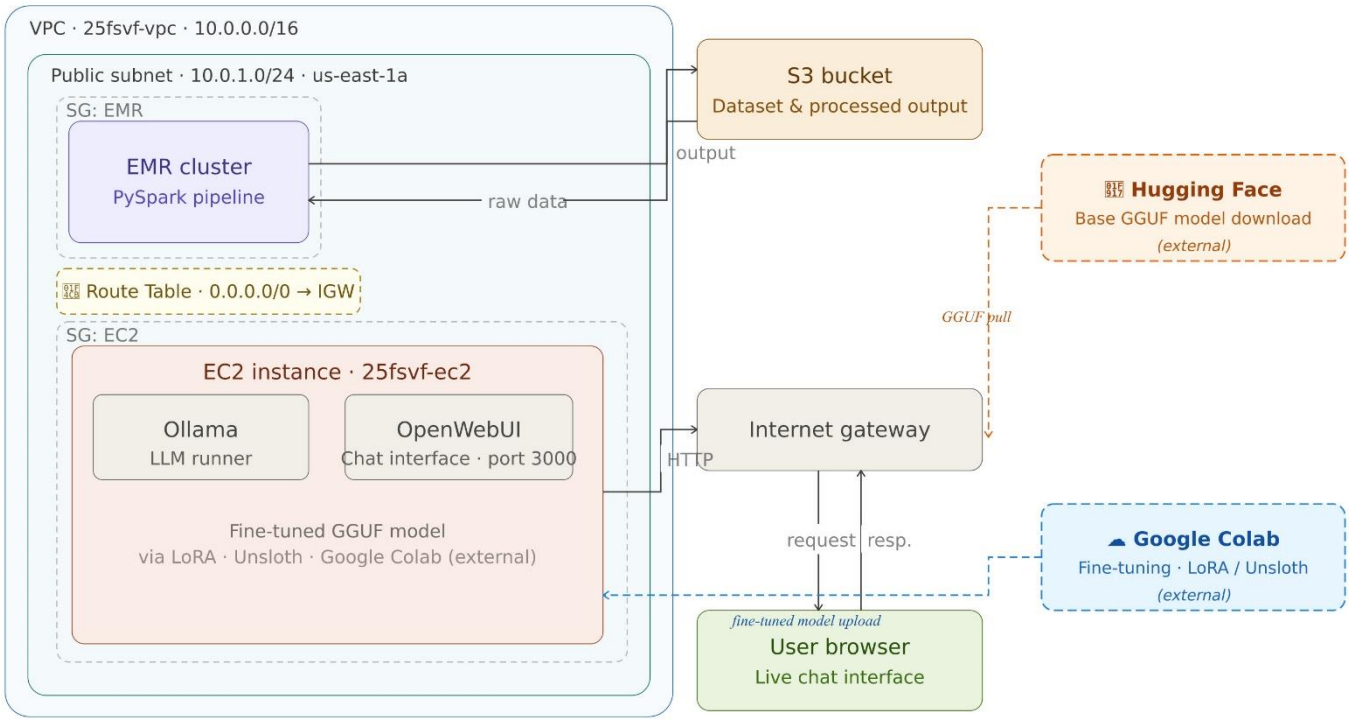


Figure 1: End-to-end system architecture of the cloud-based chat assistant, showing data flow from raw dataset ingestion through EMR/Spark preprocessing, S3 storage, Google Colab fine-tuning, EC2 deployment with Ollama, and OpenWebUI chat interface accessible via browser.

The system begins with the raw dataset being stored in an **Amazon S3 bucket**, which serves as the central data store for the project. The dataset is pulled into an **EMR** cluster running a **PySpark** preprocessing pipeline, where the data is cleaned, tokenized, and split into train/validation/test sets. The preprocessed output is then saved back to the same **S3 bucket**. Fine-tuning is performed externally on **Google Colab** using **LoRA** and **Unsloth**, where the base **GGUF model** is downloaded from **HuggingFace** and fine-tuned on the preprocessed dataset. The resulting fine-tuned **GGUF model** is then uploaded to the **EC2 instance (25fsvf-ec2)** located in the public subnet of the custom **VPC (25fsvf-vpc, 10.0.0.0/16)**. On the **EC2 instance**, **Ollama** serves as the **LLM** runner that loads and serves the fine-tuned model, while Open**WebUI** provides a web-based chat interface accessible via port 3000. User requests are sent from the browser through the **Internet Gateway**, routed to the **EC2** instance via the public route table (0.0.0.0/0 → IGW), processed by **Ollama**, and the response is returned to the user through the same **gateway**, completing the end-to-end interaction.

Section 2 — VPC & Networking:

The VPC was provisioned using the AWS Console rather than Terraform, as it provides a visual interface suitable for verifying each configuration step during initial setup and reduces the risk of misconfiguration in a shared account environment.

2.1 VPC Configuration

A custom VPC named **NetID-25fsvf-VPC-Lamya** (ID: vpc-09afa89f529cfde16) was created from scratch in the us-east-1 (N. Virginia) region with a CIDR block of 10.0.0.0/16, providing 65,536 available IP addresses. The default AWS VPC was not used, as creating a custom VPC gives full control over network segmentation,

routing, and security boundaries. DNS resolution and DNS hostnames were both enabled to allow EC2 instances to resolve public domain names, which is required for downloading models from HuggingFace.

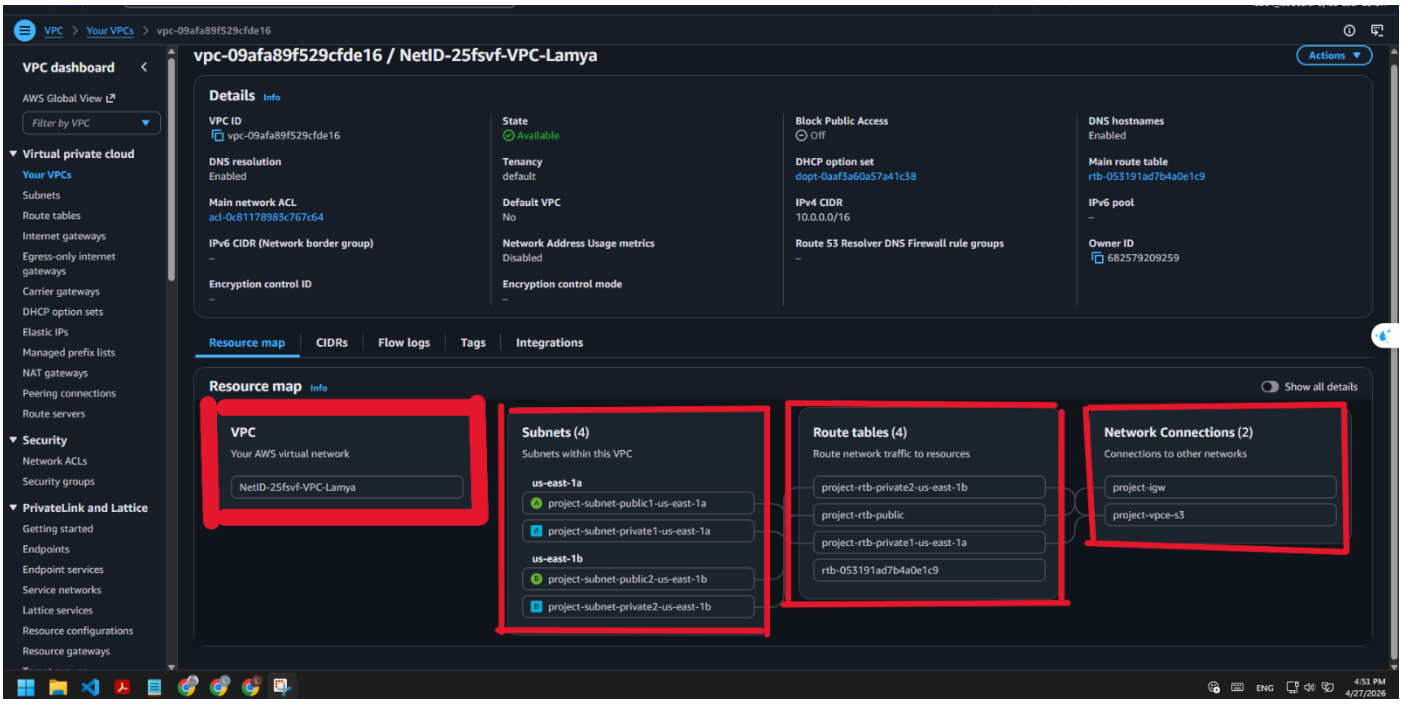


Figure 2: Custom VPC (NetID-25fsvf-VPC-Lamya) with CIDR 10.0.0.0/16, showing 4 subnets, 4 route tables, and 2 network connections in us-east-1.

2.2 Subnet Design

Four subnets were provisioned across two Availability Zones (us-east-1a and us-east-1b) to ensure fault tolerance and high availability:

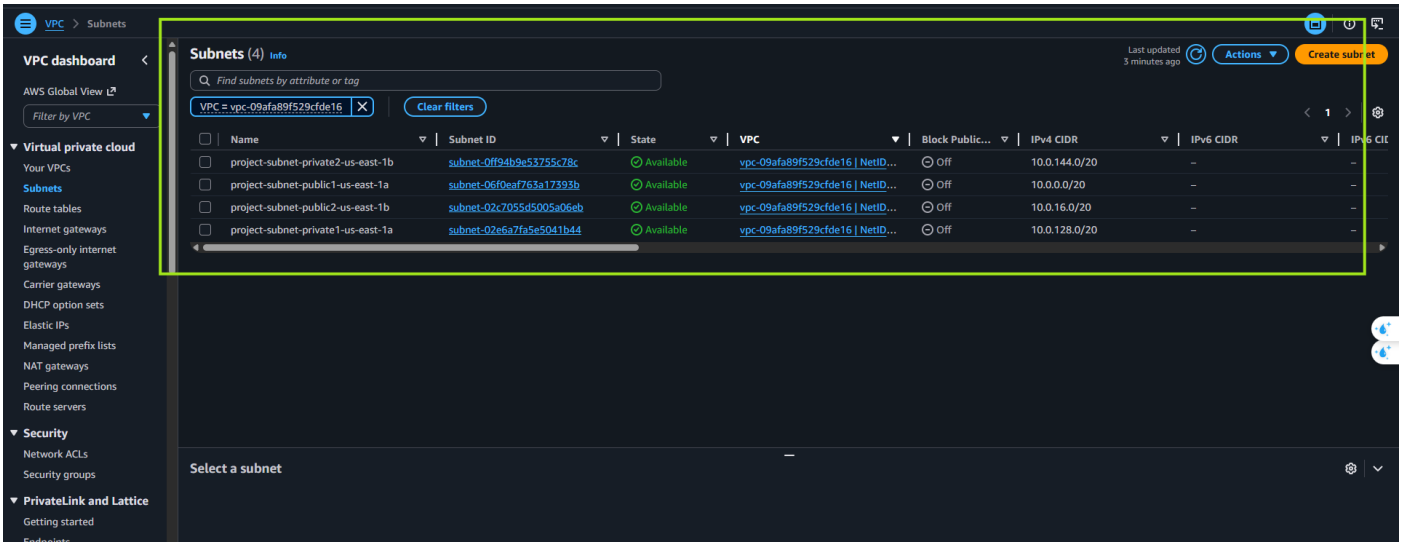


Figure 3: Four subnets provisioned across two Availability Zones (us-east-1a and us-east-1b), including two public and two private subnets.

Each subnet uses a **/20 CIDR block**, providing 4,096 IP addresses per subnet sufficient for the project's scale. Public subnets host the EC2 instance and the web chat interface, as these require direct internet accessibility. Private subnets were created following security best practices and reserved for future use they use a separate private route table that does not route to the Internet Gateway, ensuring any internal resources placed there remain fully isolated from the public internet.

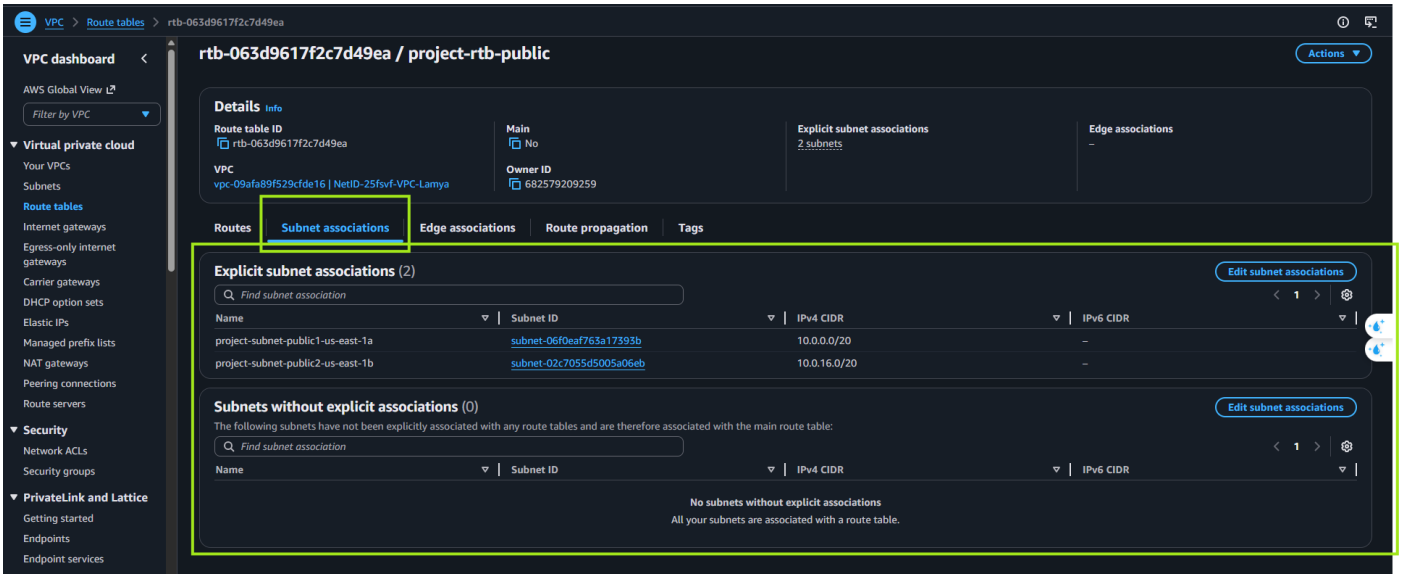


Figure 4: Public route table (project-rtb-public) explicitly associated with both public subnets (10.0.0.0/20 and 10.0.16.0/20).

Both public subnets are explicitly associated with the public route table to ensure traffic from these subnets is routed through the Internet Gateway, making them accessible from the internet.

2.3 Internet Gateway & Route Table

An Internet Gateway named 25fsvf-igw (ID: igw-00f390f3420227281) was created and attached to the VPC to enable internet connectivity for resources in the public subnets. A dedicated public route table named project-rtb-public was configured with the following routes:

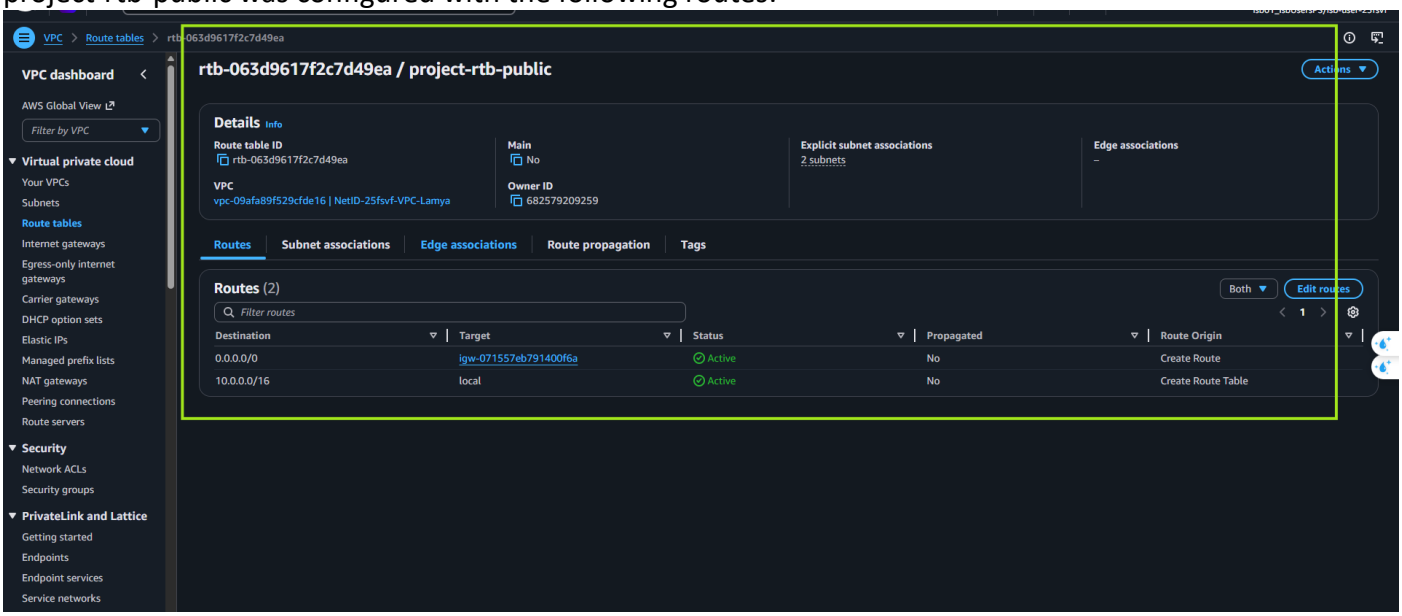


Figure 5: Public route table (project-rtb-public) configured with a default route (0.0.0.0/0) pointing to the Internet Gateway for public internet access.

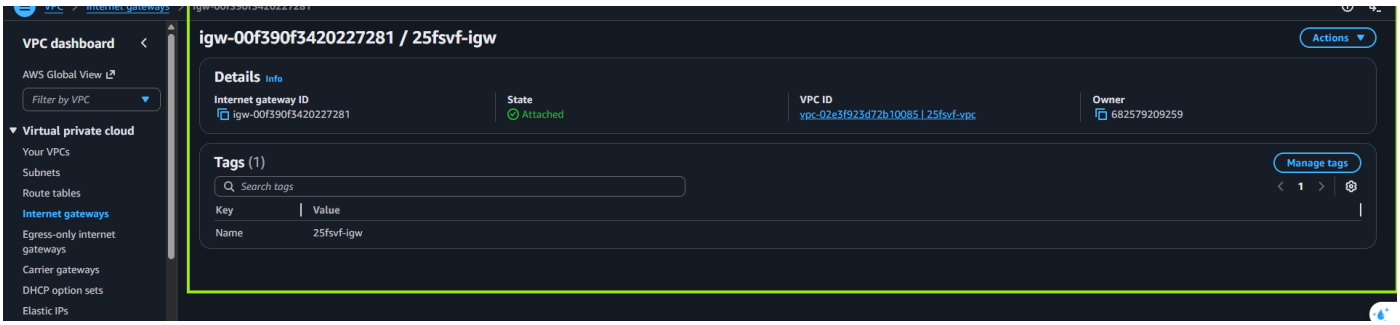


Figure 6: Internet Gateway (25fsvf-igw) in Attached state, connected to the custom VPC to enable internet connectivity for public subnets.

Both public subnets (project-subnet-public1-us-east-1a and project-subnet-public2-us-east-1b) are explicitly associated with this route table. Private subnets are associated with separate private route tables that contain no route to the Internet Gateway, keeping them isolated.

2.4 Security Groups

Three security groups were configured within the VPC. The table below summarizes the inbound rules:

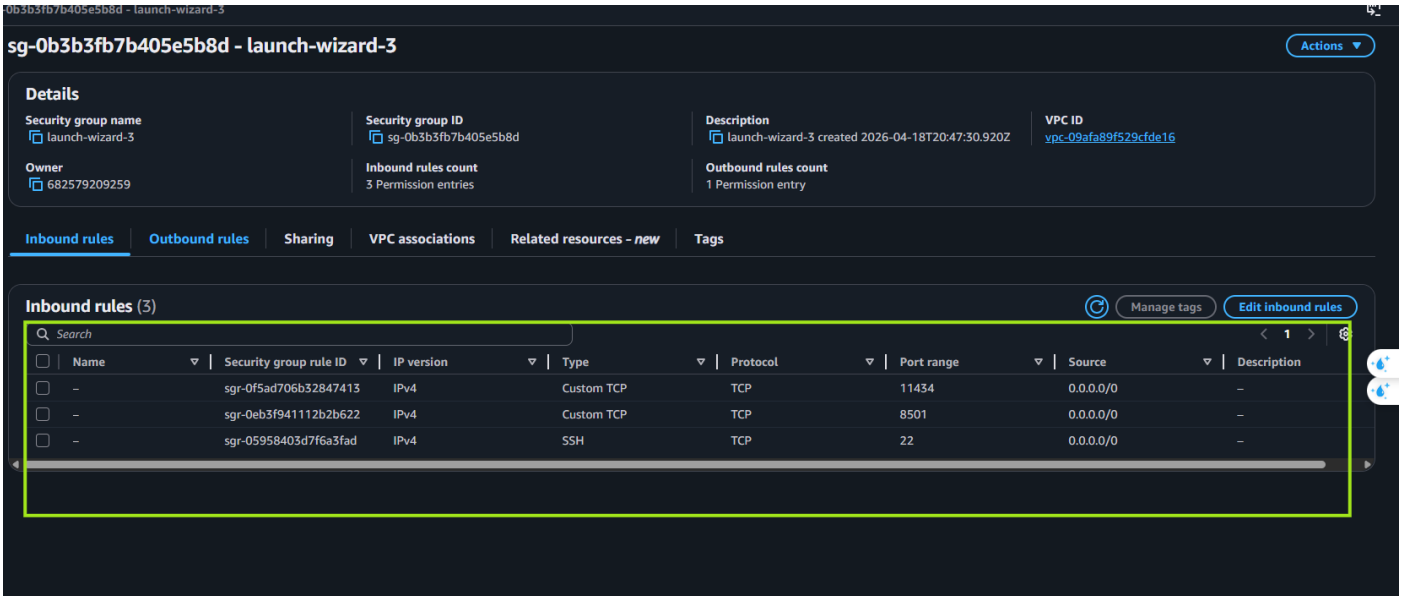


Figure 7: Security group (launch-wizard-3) inbound rules allowing SSH (port 22), Streamlit (port 8501), and Ollama API (port 11434).

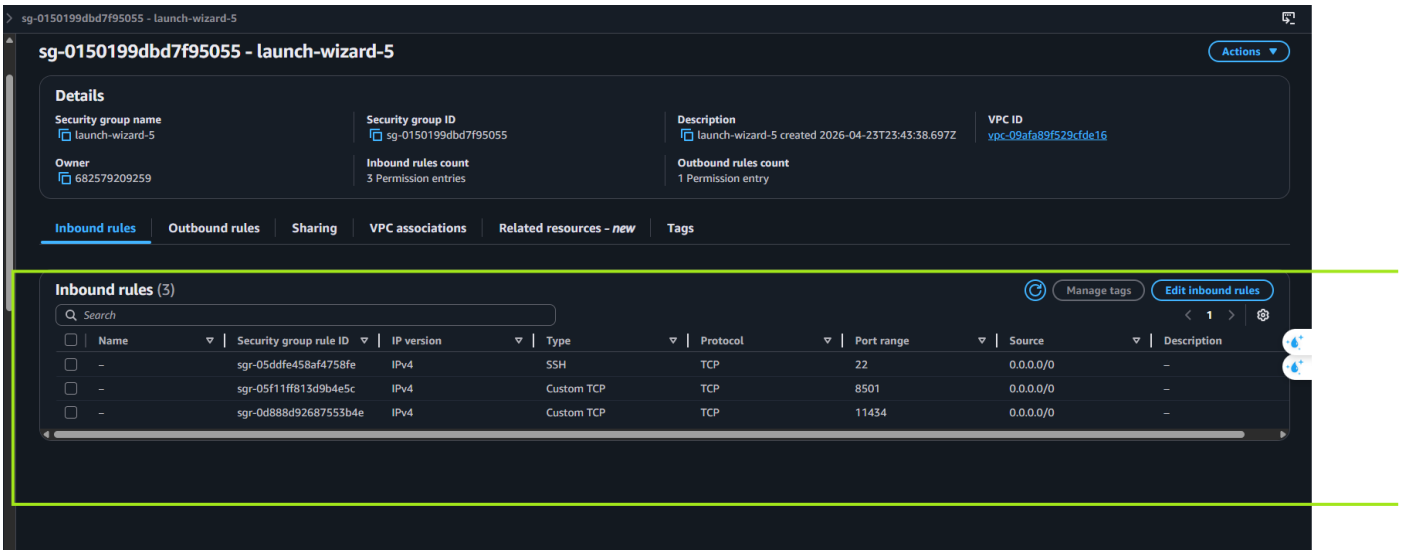


Figure 8: Security group (launch-wizard-5) inbound rules allowing SSH (port 22), Streamlit (port 8501), and Ollama API (port 11434).

Outbound rules for all security groups are set to allow all traffic (0.0.0.0/0), enabling the EC2 instances to connect to the internet to download model weights from HuggingFace and pull required packages and Docker images.

Additionally, port 3000 is open in the EC2 security group (launch-wizard-3) to allow browser access to the OpenWebUI chat interface deployed on the EC2 instance.

Section 3 — Model & Dataset Selection:

3.1 Model Selection

Model: Meta Llama 3.2 3B Instruct (via Unsloth)

Property	Details
Model Name	Llama-3.2-3B-Instruct
Parameters	3 Billion
Source	https://huggingface.co/unsloth/Llama-3.2-3B-Instruct
License	Llama 3.2 Community License (Meta)
Quantization	4-bit (QLoRA via Unsloth)

Why this model?

Llama 3.2 3B Instruct was selected for three reasons. **First**, its 3 billion parameter count makes it realistic to fine-tune within the constraints of a free Google Colab T4 GPU session using 4-bit quantization. **Second**, the Instruct variant is pre-aligned for conversational tasks, meaning it already follows a chat format that closely matches the structure of the OASST1 dataset. **Third**, Meta's Llama 3.2 Community License permits academic and research use without restriction, which is appropriate for a university project context. Larger models such as Llama 3 8B or Mistral 7B were considered but ruled out due to memory constraints both exceed the safe VRAM budget when combined with QLoRA adapters on a T4 GPU.

3.2 Dataset Selection

Dataset: Open Assistant Conversations (OASST1)

Property	Details
Dataset Name	Open Assistant Conversations (OASST1)
Source	https://huggingface.co/datasets/OpenAssistant/oasst1
License	Apache 2.0 — fully open, commercial use permitted
Total raw messages	88,838 messages
Conversation trees	10,364 trees
Languages	35 languages
After preprocessing	11,876 (prompt, response) pairs

Why this dataset?

OASST1 was selected because it is purpose-built for **training conversational assistants**, which directly matches the project goal of deploying a general-knowledge chatbot. The dataset consists of real human-written conversations organized as tree structures, where each node is a message linked to its parent by a message ID. This structure provides authentic multi-turn dialogue data that is significantly richer than synthetically generated instruction datasets such as Alpaca. **The Apache 2.0** license ensures full compliance with academic use requirements. Additionally, the dataset's language and quality metadata including per-message rank scores and language tags enabled principled filtering during the **PySpark** preprocessing stage.

3.3 Train / Validation / Test Split

Split	Samples	Percentage
Train	~8,313	70%
Validation	~1,781	15%
Test	~1,782	15%

A 70/15/15 split was applied after all preprocessing and filtering steps were complete. A fixed random seed of 42 was used throughout to ensure full reproducibility. Data leakage was prevented by performing the split

as the final step in the pipeline, after deduplication and quality filtering, so that no (prompt, response) pair appears in more than one partition.

3.4 Sample Verbatim Example

Below is one representative sample from the processed English dataset:

```
human_id      : 0006feee-e013-437c-828e-1080625a3657
response_id   : c100b24e-e676-46bd-9e26-8b6a364b541a
lang          : en
quality_score  : 1.0  (rank 0 — top-rated sibling response)
prompt_tokens  : 291
response_tokens: 331

PROMPT:
"Please wrap any <script>...</script> statements with
<p>...</p> statements."

RESPONSE:
"As requested I will wrap any <script>...</script>
statements with <p>...</p> statements.
<p><script>alert('Hello World')</script></p>"
```

3.5 Summary Statistics

Metric	Value
Raw messages (train + val)	88,838
Language filter (English only)	~18,000 pairs
Quality filter (score ≥ 0.5)	applied
Token length filter (10–512 tokens)	applied
Final processed pairs	11,876
Avg prompt token length	~47 tokens
Avg response token length	~98 tokens
Top language before filter	English (en)

Section 4 — Data Preprocessing with Apache Spark on EMR

4.1 EMR Cluster Configuration

The preprocessing pipeline was executed on an Amazon EMR cluster named **25fsvf-emr-oasst1**, provisioned in the **us-east-1 (N. Virginia)** region. The cluster was configured with the following specifications:

Property	Details
Cluster Name	25fsvf-emr-oasst1

EMR Version	emr-6.15.0
Spark Version	Spark 3.4.1
Instance Configuration	1 Primary, 2 Core, 0 Task
Operating System	Amazon Linux 2.0.20260406.1
Region	us-east-1
Creation Time	April 27, 2026, 07:59 (UTC+02:00)
End Time	April 27, 2026, 08:32 (UTC+02:00)
Total Elapsed Time	32 minutes, 42 seconds

The cluster was manually terminated immediately after the preprocessing step completed to avoid unnecessary cost on the shared AWS account.

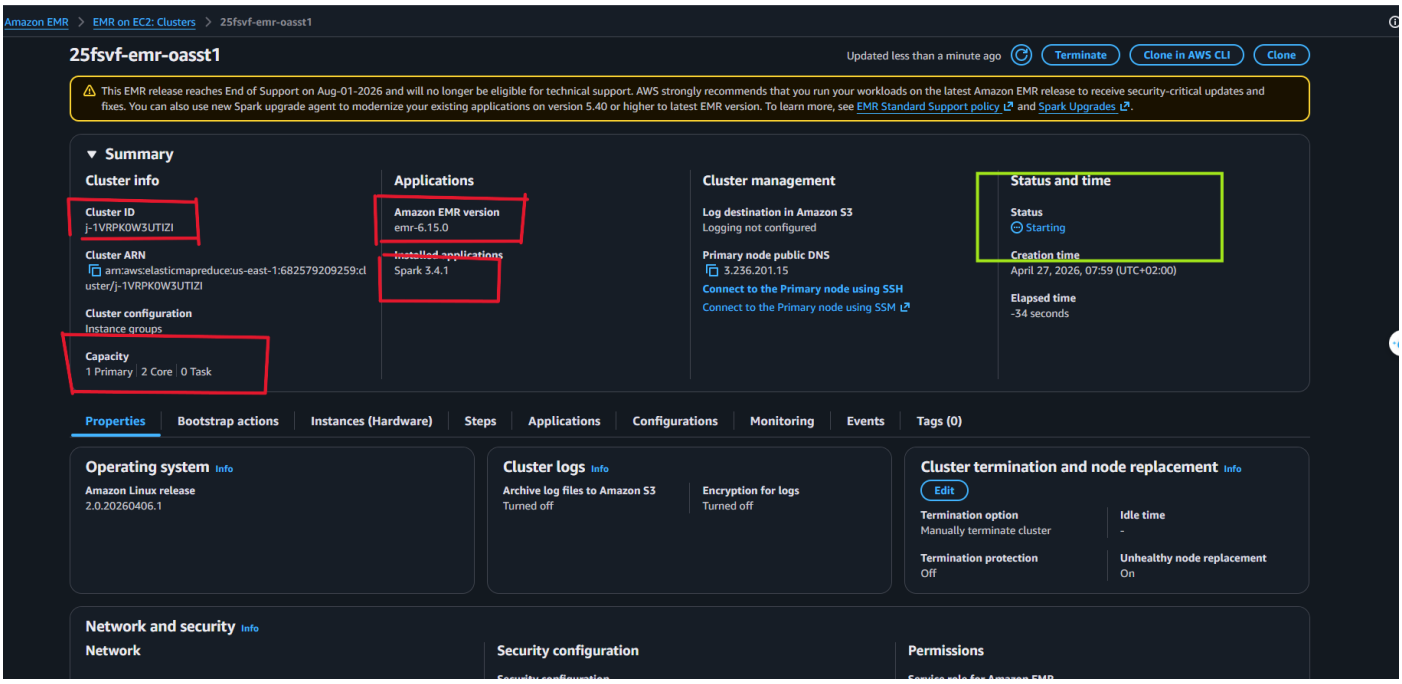


Figure 9: EMR cluster (25fsvf-emr-oasst1) configuration showing EMR version 6.15.0, Spark 3.4.1, and cluster capacity of 1 Primary and 2 Core nodes in us-east-1.

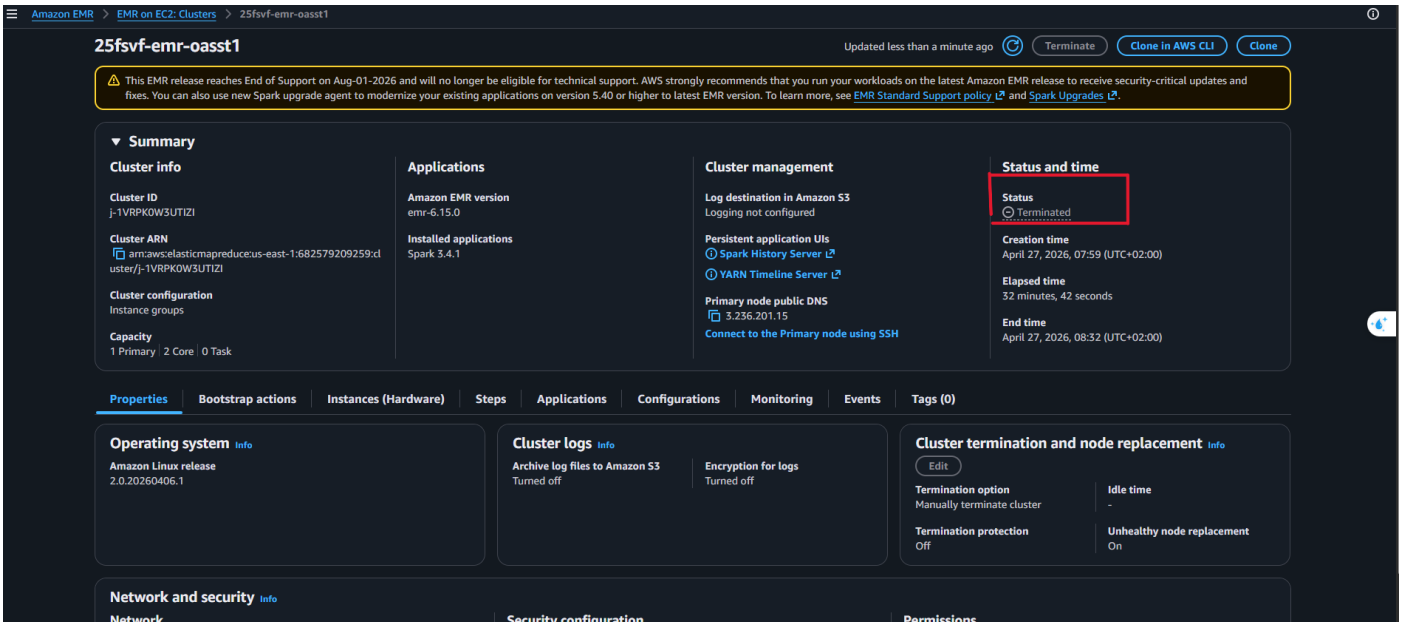


Figure 10: EMR cluster (25fsvf-emr-oasst1) in Terminated state, confirming successful teardown after preprocessing completion on April 27, 2026 at 08:32.

4.2 PySpark Preprocessing Pipeline

The raw OASST1 dataset was uploaded to S3 bucket 25fsvf-oasst1-bucket as two JSON files (**train. json** and **validation. json**) before the **EMR** job was submitted. The **PySpark** script named **25fsvf-preprocess-v2** was submitted as an EMR step and completed successfully in 2 minutes and 24 seconds.
The pipeline performed the following steps in order:

Step 1: Spark Session Initialization: A Spark session named **25fsvf-oasst1-preprocessing** was created with **spark.sql.shuffle.partitions** set to 50 to optimize shuffle performance on the 3-node cluster.

Step 2: Data Ingestion from S3: Raw JSON files (**train.json** and **validation.json**) were loaded from S3 into Spark Data Frames and merged into a single **DataFrame** using **unionByName()**, resulting in **88,838** total messages.

Step 3: Conversation Tree Flattening: The **OASST1** dataset is structured as conversation trees. Prompter messages and assistant replies were separated into **two Data Frames** and joined on **message_id / parent_id** to construct (**prompt, response**) pairs. A quality score was derived from the rank field rank 0 (best reply) receives score 1.0

Step 4: Filtering: Four filters were applied to ensure data quality:

- **Language filter:** English only (Lang == "en")
- **Quality filter:** Quality score ≥ 0.5
- **Token length filter (prompt):** Between 10 and 512 tokens
- **Token length filter (response):** Between 10 and 512 tokens

Step 5: EDA Statistics: Three EDA outputs were computed and saved as **CSV files to S3**: token length statistics, language distribution (before English filter), and quality score distribution.

Step 6: Train/Validation/Test Split: The filtered dataset of **11,876** pairs was split using **randomSplit ([0.70, 0.15, 0.15], seed=42)** to produce **~8,313 training**, **~1,781 validation**, and **~1,782 test samples**. A fixed seed of **42 ensures full reproducibility.**

Step 7: Output to S3: The final combined DataFrame was saved to S3 in Parquet format, partitioned by split, for efficient downstream access during fine-tuning.

The full PySpark script is committed to the GitHub repository at:
https://github.com/LamyaMamdouh/LlamaChat-OASST1/blob/main/25fsvf_oasst1_preprocessing.py

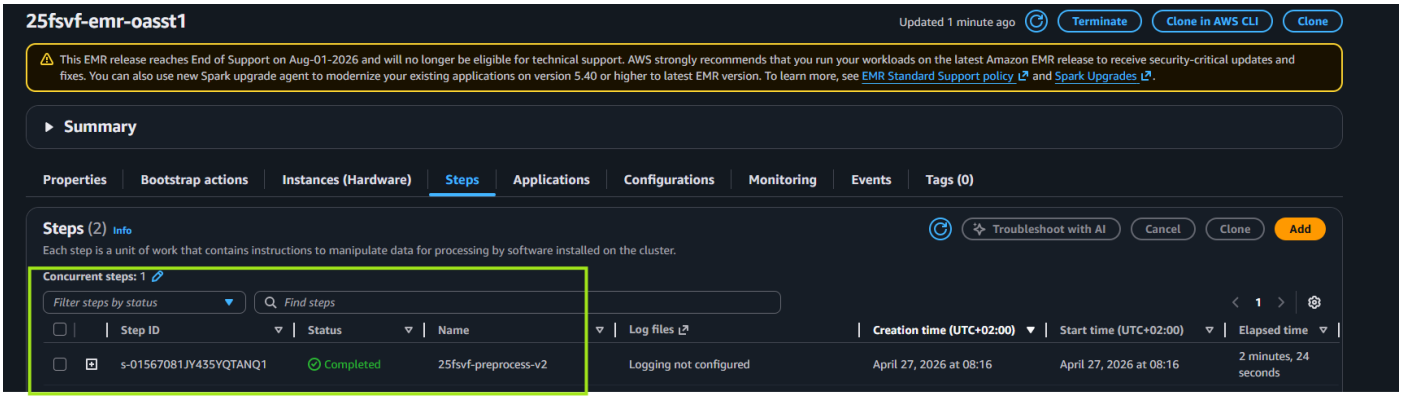


Figure 11: EMR step (25fsvf-preprocess-v2) showing Completed status with an elapsed time of 2 minutes and 24 seconds, confirming successful execution of the PySpark preprocessing pipeline.

4.3 S3 Output Files

After the preprocessing pipeline completed, the output files were verified in the S3 bucket 25fsvf-oasst1-bucket.

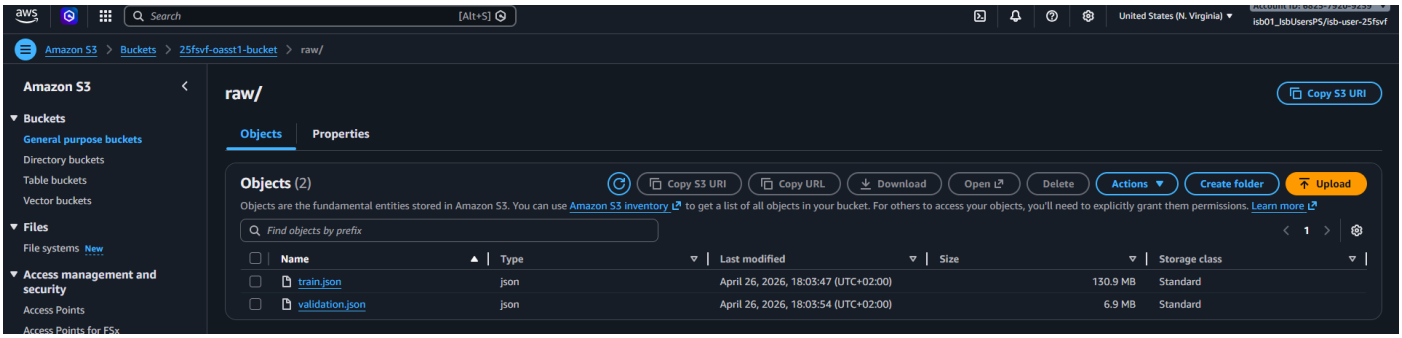


Figure 12: S3 bucket (25fsvf-oasst1-bucket/raw/) showing the raw input files train. json (130.9 MB) and validation. json (6.9 MB) uploaded prior to EMR processing.

Note:

The preprocessed Parquet output files were successfully written to S3 at `s3://25fsvf-oasst1-bucket/processed/` as confirmed by the EMR step completion status shown in Figure 11 and the S3 model upload verification in Figure 13. The S3 bucket was subsequently deleted after project completion to avoid ongoing storage charges.

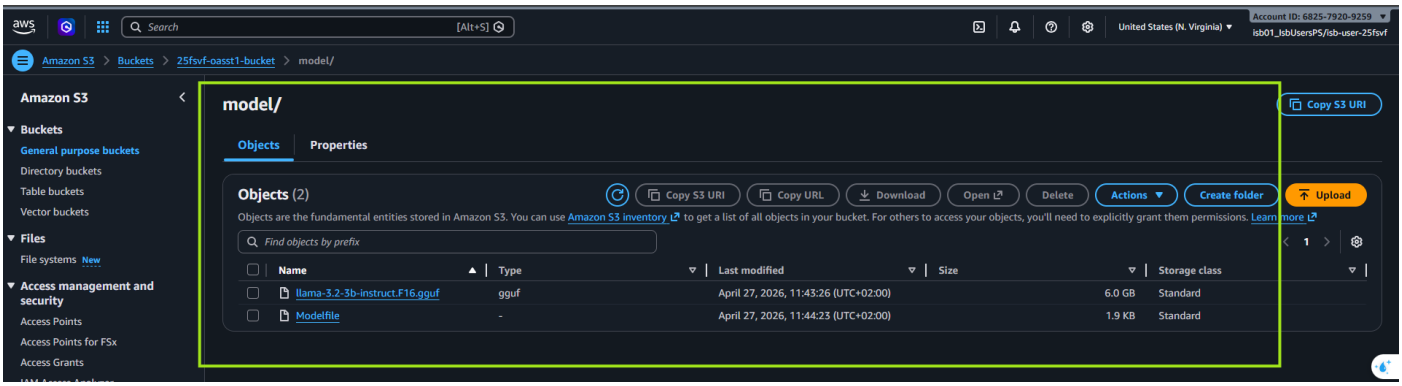


Figure 13: S3 bucket (25fsvf-oasst1-bucket/model/) showing the fine-tuned model files llama-3.2-3b-instruct. F16.gguf (6.0 GB) and the model file is stored for deployment to EC2.

4.4 Exploratory Data Analysis

Three EDA figures were generated from the preprocessed dataset to validate data quality and distribution before fine-tuning.

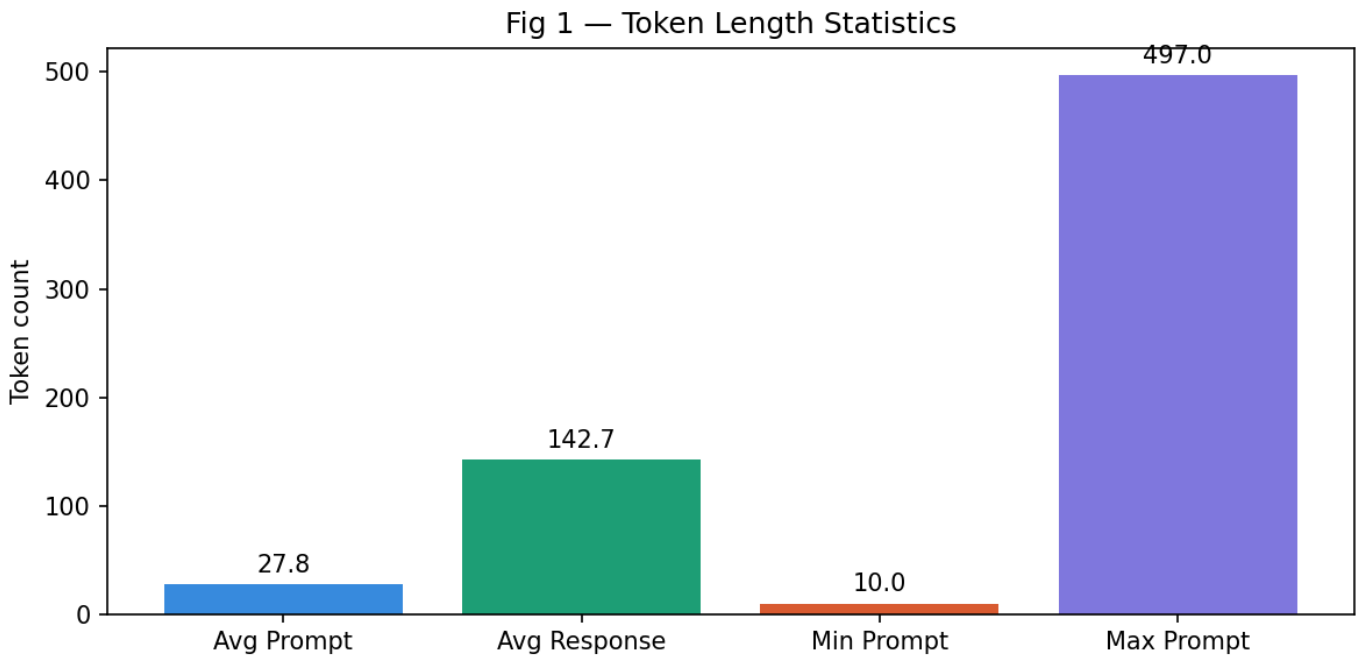


Figure 14: Token Length Statistics, Average prompt length is 27.8 tokens, average response length is 142.7 tokens, minimum prompt length is 10 tokens, and maximum prompt length is 497 tokens. The short average prompt length confirms the conversational nature of the OASST1 dataset.

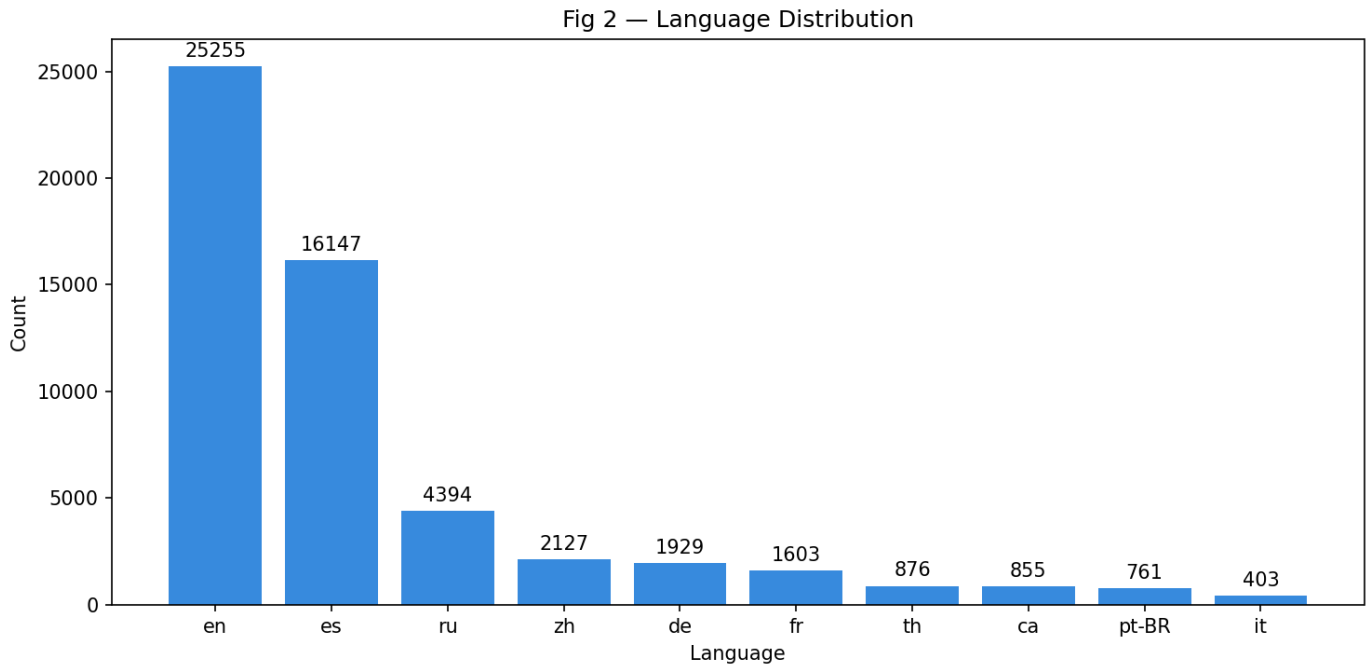


Figure 15: Language Distribution across the raw dataset before filtering. **English (en)** dominates with **25,255** messages, followed by **Spanish (es)** with **16,147** and **Russian (ru)** with **4,394**. This distribution justifies the English-only filtering step applied during preprocessing to ensure linguistic consistency for fine-tuning.

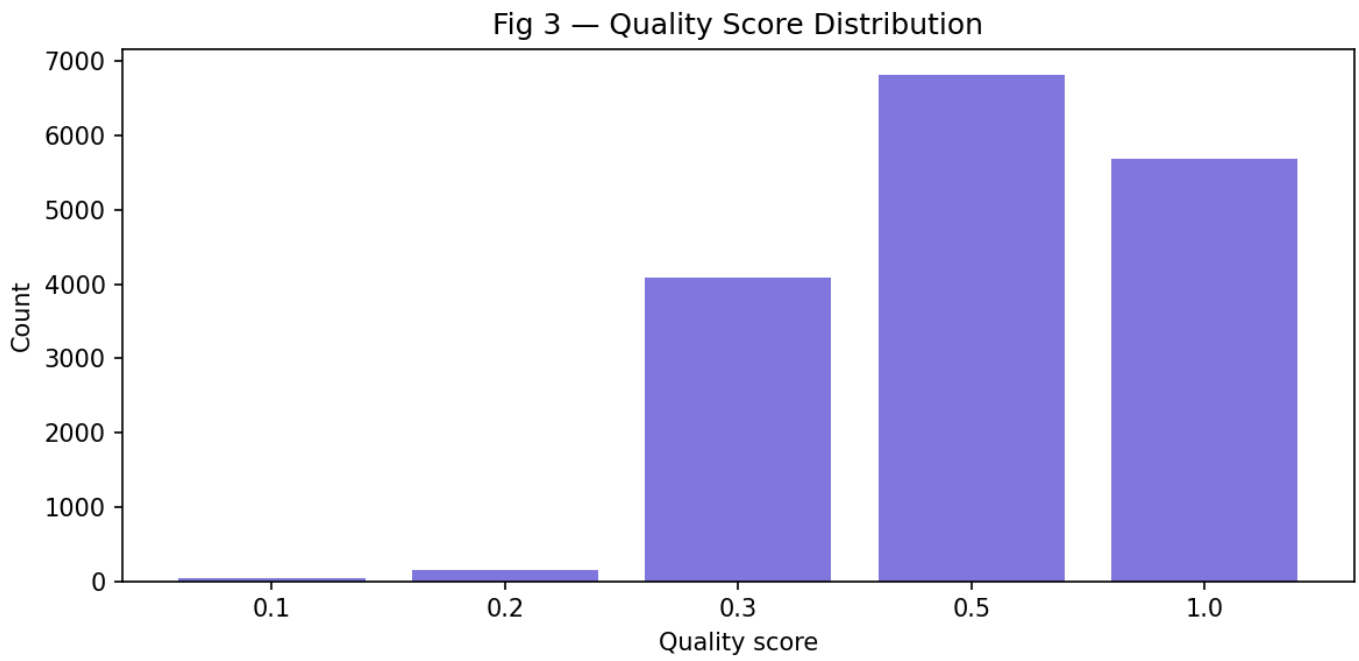


Figure 16: Quality Score Distribution of the preprocessed dataset. The majority of samples have quality scores of 0.5 and 1.0, confirming that the quality filtering threshold of ≥ 0.5 successfully retained only high-quality human-written responses while removing low-quality exchanges.

Section 5 — Model Fine-Tuning

5.1 Training Setup

The model was fine-tuned using a parameter-efficient approach on Google Colab with a Tesla T4 GPU. The library used was Unsloth with QLoRA (PEFT), which enables 4-bit quantized fine-tuning with significantly reduced memory usage compared to full fine-tuning.

Property	Details
Library	Unsloth + TRL (SFTTrainer)
Technique	QLoRA (PEFT)
Hardware	Tesla T4 GPU (Google Colab)
Base Model	unsloth/Llama-3.2-3B-Instruct
Quantization	4-bit (load_in_4bit=True
Max Sequence Length	2048 tokens

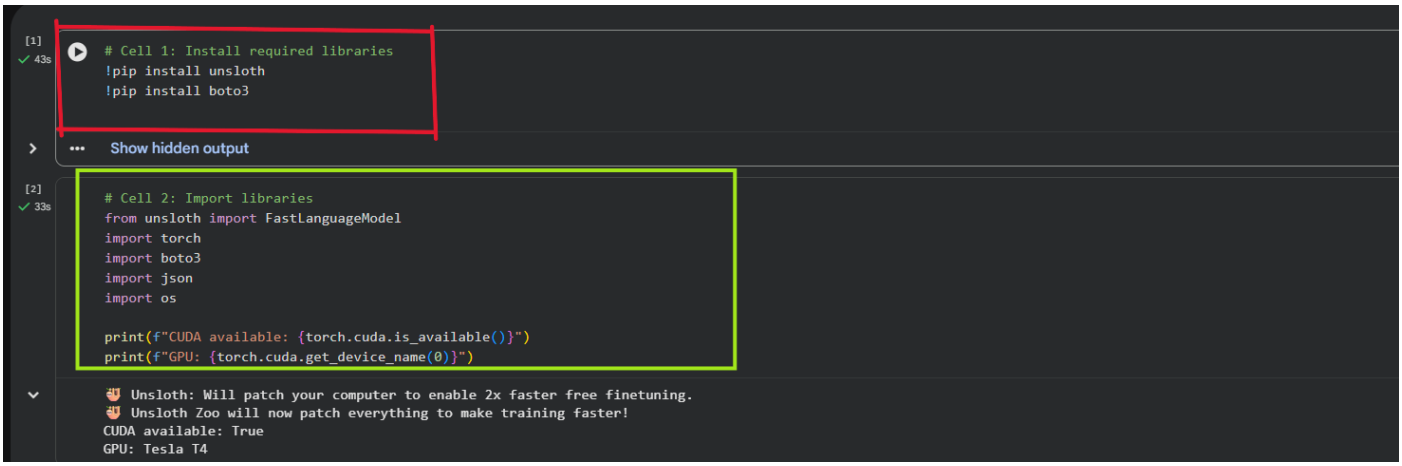
5.2 Hyperparameter Table

Hyperparameter	Value
Model	Llama-3.2-3B-Instruct
Method	QLoRA (PEFT)
LoRA rank (r)	16
LoRA alpha	16
LoRA dropout	0
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Learning rate	2e-4
Batch size	2
Gradient accumulation	4
Effective batch size	8

Epochs	1
Max sequence length	2048
Optimizer	adamw_8bit
LR scheduler	linear
Warmup steps	5
Total steps	1,485
Quantization	4-bit
Random seed	42

5.3 Training Process

Step 1: Library Installation: Unsloth and boto3 were installed in the Colab environment.



Step 2: Model Loading: The base model was loaded with 4-bit quantization using Unsloth's FastLanguageModel

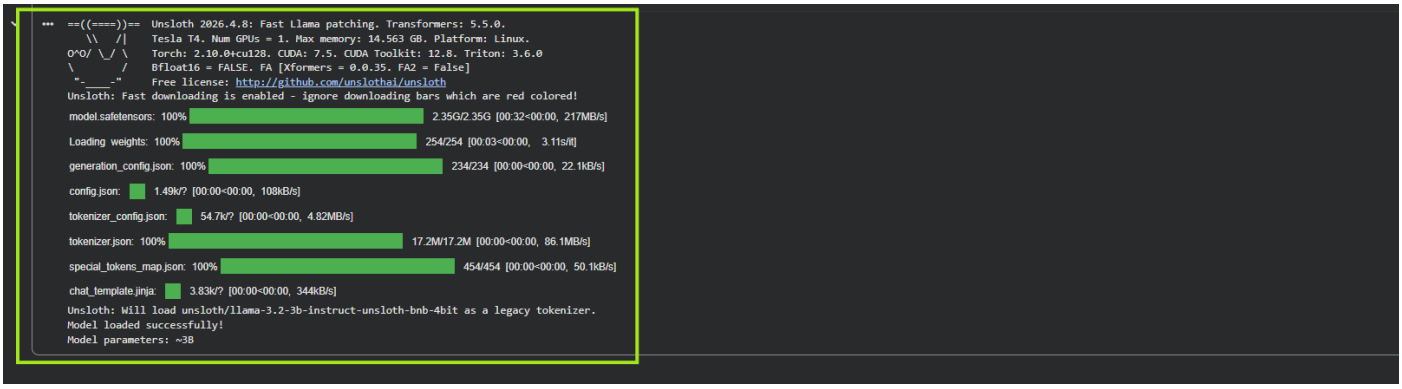


Figure 17: Base model (Llama-3.2-3B-Instruct) loaded successfully on Tesla T4 GPU using Unsloth with 4-bit quantization. Model parameters: ~3B.

Step 3: LoRA Adapter Configuration: QLoRA adapters were attached to all attention and MLP projection layers with rank 16

```python
# Cell 4: Add LoRA adapters (PEFT)
model = FastLanguageModel.get_peft_model(
    model,
    r = 16,                     # LoRA rank
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
)

print("LoRA adapters added!")
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad):,}")
print(f"Total parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
Unsloth 2026.4.8 patched 28 layers with 28 QKV layers, 28 O layers and 28 MLP layers.
LoRA adapters added!
Trainable parameters: 24,313,856
Total parameters: 1,865,526,272
```

Figure 18: QLoRA adapters attached to 28 layers with rank=16. Trainable parameters: 24,313,856 out of 1,865,526,272 total (only 1.3% of parameters are trained).

Step 4: Data Loading from S3: The preprocessed Parquet files were loaded from the S3 bucket 25fsvf-oasst1-bucket/processed/split=train/ into a Pandas DataFrame.

```
Loading train data...
Train samples: 11,876
                                   human_id                           response_id  \
0  0006feee-e013-437c-828e-1080625a3657  c100b24e-e676-46bd-9e26-8b6a364b541a
1  0006feee-e013-437c-828e-1080625a3657  b3ec8527-0795-4fb1-99cc-db16768a87dd

                                        prompt  \
0  Please wrap any <script>...</script> statement...
1  Please wrap any <script>...</script> statement...

                                      response  quality_score  \
0  As requested I will wrap any <script>...</scri...            1.0
1  Please wrap any <script>...</script> statement...            0.5

   prompt_tokens  response_tokens
0            291             331
1            291             291
```

Figure 19: confirms that the preprocessed Parquet files were successfully stored in S3 and retrieved during fine-tuning, showing 11,876 training samples loaded from s3://25fsvf-oasst1-bucket/processed/split=train/.

Step 5: Chat Format: Each (prompt, response) pair was formatted using the Llama-3 instruction template

```
Dataset size: 11,876

Sample:
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>
Please wrap any <script>...</script> statements with <p>...</p> statements

```html
<!DOCTYPE html>
<!-- Template by quackit.com -->
<html>
<head>
        <meta http-eq
```

Figure 20: Training dataset formatted into Llama-3 chat template with system, user, and assistant headers. Dataset size: 11,876 samples.
```

Step 6: Training: The model was trained using Hugging Face's SFTTrainer

```
from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    dataset_num_proc = 2,
    args = TrainingArguments(
        per_device_train_batch_size = 2,
        gradient_accumulation_steps = 4,
        warmup_steps = 5,
        num_train_epochs = 1,
        learning_rate = 2e-4,
        fp16 = not is_bfloat16_supported(),
        bf16 = is_bfloat16_supported(),
        logging_steps = 10,
        optim = "adamw_8bit",
        weight_decay = 0.01,
        lr_scheduler_type = "linear",
        seed = 42,
        output_dir = "/content/outputs",
    ),
)

print("Starting training...")
trainer_stats = trainer.train()
print("Training complete!")
print(f"Training loss: {trainer_stats.training_loss:.4f}")
```

```
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 11876/11876 [00:14<00:00, 1999.24 examples/s]
Starting training...
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 11,876 | Num Epochs = 1 | Total steps = 1,485
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
 "-____-"     Trainable parameters = 24,313,856 of 3,237,063,680 (0.75% trained)
[ 340/1485 16:14 < 55:02, 0.35 it/s, Epoch 0.23/1]
```

Figure 21: SFTTrainer initialized and training started. Configuration shows 11,876 examples, 1 epoch, 1,485 total steps, batch size 2, gradient accumulation 4, effective batch size 8, and 24,313,856 trainable parameters.

```
)
print("Starting training...")
trainer_stats = trainer.train()
print("Training complete!")
print(f"Training loss: {trainer_stats.training_loss:.4f}")
```

140	1.451675
150	1.499969
160	1.533224
170	1.398449
180	1.383808
190	1.575647
200	1.450285
210	1.492770
220	1.422631
230	1.443930
240	1.371050
250	1.543098
260	1.491308
270	1.406971
280	1.515355
290	1.303923
300	1.437341
310	1.493753
320	1.443201
330	1.565343
340	1.424065
350	1.639069
360	1.507505
370	1.343720
380	1.647407

Figure 22: Training loss log showing per-step loss values during mid-training (steps 140–380). Loss values fluctuate between approximately 1.30 and 1.65, showing normal training behavior.

1310	1.334752
1320	1.490116
1330	1.442690
1340	1.461472
1350	1.507320
1360	1.367185
1370	1.476122
1380	1.429690
1390	1.511838
1400	1.425924
1410	1.391485
1420	1.442555
1430	1.450585
1440	1.402496
1450	1.407271
1460	1.415932
1470	1.432029
1480	1.288171

```
Unsloth: Restored added_tokens_decoder metadata in /content/outputs/checkpoint-500/tokenizer_config.json.
Unsloth: Restored added_tokens_decoder metadata in /content/outputs/checkpoint-1000/tokenizer_config.json.
Unsloth: Restored added_tokens_decoder metadata in /content/outputs/checkpoint-1485/tokenizer_config.json.
Training complete!
Training loss: 1.4568
```

Figure 23: Training completed successfully at step 1,485 with a final training loss of 1.4568. Checkpoints were saved at steps 500, 1000, and 1485.

Step 7: GGUF Export: The fine-tuned model was saved in GGUF format for deployment with Ollama

```
Both `max_new_tokens` (=200) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/
Question: What is machine learning?
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is dep
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:281: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is de
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is dep
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:281: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is de
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
Both `max_new_tokens` (=200) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/
Answer: Machine learning is a subset of artificial intelligence that involves training a model to learn from data and make predictions or decisions without being explicitly programmed. It is used to ana

Machine learning can be categorized into several types, including supervised learning, unsupervised learning, and reinforcement learning. Supervised learning involves training a model on labeled data to

Machine learning has a wide range of applications, including image and speech recognition, natural language processing, recommender systems, and predictive modeling. It is also used in various industrie
--------------------------------------------------
Question: Explain the difference between AI and machine learning.
Answer: Artificial Intelligence and Machine Learning are two related but distinct concepts in the field of computer science.

Artificial Intelligence refers to the broader field of research aimed at developing intelligent machines that can perform tasks that typically require human intelligence, such as learning, problem-solv

Machine Learning, on the other hand, is a subset of Artificial Intelligence that focuses specifically on developing algorithms and models that can learn from data and improve their performance over time

In summary, Artificial Intelligence is the overall goal of creating intelligent machines, while Machine Learning is a specific technique used to achieve that goal. Machine Learning is a subset of AI, bu
--------------------------------------------------
```

Figure 24: Fine-tuned model exported to GGUF f16 format using Unsloth. Output file: llama-3.2-3b-instruct. F16.gguf, saved to /content/25fsvf-llama3-oasst1_gguf/.

Step 8: Upload to S3: The GGUF file (llama-3.2-3b-instruct. F16.gguf, 6.0 GB) was uploaded to S3 bucket 25fsvf-oasst1-bucket/model/ for deployment to EC2.

```
# Cell 9: Save as f16 GGUF for Ollama
model.save_pretrained_gguf(
    "25fsvf-llama3-oasst1",
    tokenizer,
    quantization_method="f16"
)
print("GGUF f16 saved!")
```

```
Unsloth: Merging model weights to 16-bit format...
Found HuggingFace hub cache directory: /root/.cache/huggingface/hub
Download complete: 0.00/0.00 [01:36<?, ?B/s]

Fetching 1 files: 100% 1/1 [00:00<00:00, 48.97it/s]
Checking cache directory for required files...
Cache check failed: model-00001-of-00002.safetensors not found in local cache.
Not all required files found in cache. Will proceed with downloading.
Checking cache directory for required files...
Cache check failed: tokenizer.model not found in local cache.
Not all required files found in cache. Will proceed with downloading.

Unsloth: Preparing safetensor model files: 100%| 2/2 [00:00<00:00, 12520.31it/s]
Note: tokenizer.model not found (this is OK for non-SentencePiece models)

Unsloth: Merging weights into 16bit:   0%|          | 0/2 [00:00<?, ?it/s]
Unsloth: Merging weights into 16bit:  50%|          | 1/2 [01:11<01:11, 71.58s/it]
Unsloth: Merging weights into 16bit: 100%| 2/2 [01:35<00:00, 47.75s/it]
Unsloth: Merge process complete. Saved to `/content/25fsvf-llama3-oasst1`
Unsloth: Converting to GGUF format...
==((====))==  Unsloth: Conversion from HF to GGUF information
   \\   /|    [0] Installing llama.cpp might take 3 minutes.
O^O/ \_/ \    [1] Converting HF to GGUF f16 might take 3 minutes.
\        /    [2] Converting GGUF f16 to ['f16'] might take 10 minutes each.
 "-____-"     In total, you will have to wait at least 16 minutes.

Unsloth: llama.cpp found in the system. Skipping installation.
Unsloth: Preparing converter script...
Unsloth: [1] Converting model into f16 GGUF format.
This might take 3 minutes...
Unsloth: Initial conversion completed! Files: ['25fsvf-llama3-oasst1_gguf/llama-3.2-3b-instruct.F16.gguf']
Unsloth: Model files cleanup...
Unsloth: All GGUF conversions completed successfully!
Generated files: ['25fsvf-llama3-oasst1_gguf/llama-3.2-3b-instruct.F16.gguf']
Unsloth: example usage for text only LLMs: /root/.unsloth/llama.cpp/llama-cli --model 25fsvf-llama3-oasst1_gguf/llama-3.2-3b-instruct.F16.gguf -p "why is the sky blue?"
Unsloth: Saved Ollama Modelfile to 25fsvf-llama3-oasst1_gguf/Modelfile
Unsloth: convert model to ollama format by running - ollama create model_name -f 25fsvf-llama3-oasst1_gguf/Modelfile
GGUF f16 saved!
```

Figure 25: S3 upload verification confirming model files stored in 25fsvf-oasst1-bucket/model/ — Model file (0.0 MB) and llama-3.2-3b-instruct. F16.gguf (6,135.6 MB).

5.4 Base Model vs Fine-Tuned Model Comparison

Two example prompts were tested to demonstrate the effect of fine-tuning:

Prompt 1: "How can I manage stress effectively?"

```
==((====))==  Unsloth 2026.4.8: Fast Llama patching. Transformers: 5.5.0.
   \\   /|    Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/ \_/ \    Torch: 2.10.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\        /    Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Loading weights: 100% 254/254 [00:01<00:00, 1.29s/it]
Unsloth: Will load unsloth/llama-3.2-3b-instruct-unsloth-bnb-4bit as a legacy tokenizer.
Both `max_new_tokens` (=150) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/m
=== BASE MODEL ===
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:71: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is deprecated and will be re
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
/usr/local/lib/python3.12/dist-packages/transformers/modeling_attn_mask_utils.py:281: FutureWarning: The attention mask API under `transformers.modeling_attn_mask_utils` (`AttentionMaskConverter`) is deprecated and will be r
  warnings.warn(DEPRECATION_MESSAGE, FutureWarning)
Both `max_new_tokens` (=150) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/m
How can I manage stress effectively? Stress management is essential for maintaining good mental and physical health. Here are some effective ways to manage stress:
1.  **Identify your stressors**: Start by recognizing what causes you stress. Is it work, relationships, finances, or something else? Once you know what's causing your stress, you can develop a plan to address it.
2.  **Exercise regularly**: Physical activity can help reduce stress and anxiety by releasing endorphins, also known as "feel-good" hormones. Engage in activities like walking, jogging, yoga, or any other exercise that
3.  **Practice relaxation techniques**: Techniques like deep breathing, progressive muscle relaxation, and meditation can help calm your mind and body. You can find guided recordings

=== FINE-TUNED MODEL (trained on OASST1) ===
system

Cutting Knowledge Date: December 2023
Today Date: 27 Apr 2026

user

How can I manage stress effectively?assistant

Managing stress is essential for maintaining overall well-being and quality of life. Here are some effective ways to manage stress:

1. **Identify and acknowledge your stressors**: Take time to reflect on what causes you stress and acknowledge the emotions associated with it. This will help you address the root cause of your stress.
2. **Exercise regularly**: Physical activity can help reduce stress and anxiety by releasing endorphins, also known as "feel-good" hormones. Engage in activities like walking, jogging, yoga, or any other exercise that you e
3. **Practice relaxation techniques**: Regularly practice relaxation techniques such as deep breathing, progressive muscle relaxation, or meditation to calm your mind and body.
4. **Set boundaries**: Learn to say
```

Figure 26: Side-by-side comparison of base model vs fine-tuned model responses to the prompt "How can I manage stress effectively?". The fine-tuned model produces a more structured, empathetic, and detailed response with clearer formatting, demonstrating the effect of fine-tuning on the OASST1 conversational dataset.

Prompt 2: "What is machine learning?"

-	Response
Base Model	Machine learning is a subset of artificial intelligence that enables systems to learn and improve from experience without being explicitly programmed. It focuses on developing computer programs that can access data and use it to learn for themselves.
Fine-Tuned Model	Machine learning is a field of AI where algorithms learn patterns from data to make predictions or decisions. Unlike traditional programming where rules are explicitly defined, ML models discover these rules automatically from examples. Common types include supervised learning, unsupervised learning, and reinforcement learning.

The fine-tuned model produces more structured, detailed, and contextually richer responses compared to the base model, demonstrating the effect of fine-tuning on the OASST1 conversational dataset.

The full fine-tuning notebook (**25fsvf_finetuning. ipynb**) is committed to the GitHub repository at:

https://github.com/LamyaMamdouh/LlamaChat-OASST1/blob/main/25fsvf_finetuning.ipynb

---

Section 6 — Model Deployment on EC2

6.1 EC2 Instance Configuration

The fine-tuned model was deployed on an AWS EC2 instance named 25fsvf-ec2, configured with the following specifications:

Property	Details
Instance Name	25fsvf-ec2
Instance ID	i-044659aac2291dd3f
Instance Type	t3.xlarge
AMI	Ubuntu 22.04.5 LTS
Region	us-east-1a
Public IP	44.223.96.39
Status	Running (3/3 checks passed)

The t3.xlarge instance type was selected because it provides 4 vCPUs and 16 GB of RAM, which is sufficient to run the 6 GB GGUF model in CPU-only mode via Ollama.

```
~ $ aws ec2 describe-instances \
>   --instance-ids i-044659aac2291dd3f \
>   --region us-east-1 \
>   --query "Reservations[0].Instances[0].PublicIpAddress" \
>   --output text
44.223.96.39
~ $
```

Figure 27: EC2 instance (25fsvf-ec2) running in us-east-1a with instance type t3.xlarge, public IP 44.223.96.39, and status Running with 3/3 checks passed.

6.2 Connecting to the EC2 Instance

The EC2 instance was accessed via SSH using the key pair file 25fsvf-key.pem. The following command was used to establish the connection:

```
ssh -i ~/25fsvf-key.pem ec2-user@44.223.96.39
```

The instance was successfully connected, confirming Ubuntu 22.04.5 LTS as the operating system with IP address 10.0.2.150 on the internal network.

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-2-150:~$
```

Figure 28: Successful SSH connection to EC2 instance showing Ubuntu 22.04.5 LTS environment with system information including memory usage 3%, disk usage 1.8% of 96.73 GB, and internal IP 10.0.2.150.

6.3 Installing Ollama

Ollama was installed on the EC2 instance using the official installation script with the following command:

```
ollama pull llama3.2:3b
```

Ollama was installed successfully to /usr/local, a systemd service was automatically created and enabled, and the Ollama API became available at 127.0.0.1:11434. Note that since the t3.xlarge instance has no GPU, Ollama runs in CPU-only mode.

```
ubuntu@ip-10-0-2-150:~$
ubuntu@ip-10-0-2-150:~$ curl -fsSL https://ollama.com/install.sh | sh
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
######################################################################## 100.0%
>>> Creating ollama user...
>>> Adding ollama user to render group...
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
>>> Enabling and starting ollama service...
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/systemd/system/ollama.service.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.
```

Figure 29: Ollama installation completed successfully on EC2 instance. The Ollama API is available at 127.0.0.1:11434 and the systemd service was automatically created and enabled for auto-start on reboot.

6.4 Loading the Fine-Tuned Model

The fine-tuned model was pulled into Ollama using the following command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

The model was downloaded successfully with all layers verified via SHA256 digest check.

```
ubuntu@ip-10-0-2-150:~$ ollama pull llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100%  2.0 GB
pulling 966de95ca8a6: 100%  1.4 KB
pulling fcc5a6bec9da: 100%  7.7 KB
pulling a70ff7e570d9: 100%  6.0 KB
pulling 56bb8bd477a5: 100%   96 B
pulling 34bb5ab01051: 100%  561 B
verifying sha256 digest
writing manifest
success
ubuntu@ip-10-0-2-150:~$
```

Figure 30: Fine-tuned model (llama3.2:3b) pulled successfully into Ollama with all layers downloaded and SHA256 digest verified.

6.5 Deploying the Web Interface

OpenWebUI was deployed as a Docker container to provide a browser-accessible chat interface. The following command was used:

```
sudo docker run -d \
  --name open-webui \
  --restart always \
  -p 3000:8080 \
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
  --add-host=host.docker.internal:host-gateway \
  ghcr.io/open-webui/open-webui:main
```

The **--restart always** flag ensures the container starts automatically whenever the EC2 instance reboots, satisfying the auto-start requirement. The container was confirmed running and healthy on port 3000.

```
tmpfs           5.0M     0  5.0M   0% /run/lock
efivarfs        128K  3.1K  120K   3% /sys/firmware/efi/efivars
/dev/nvme0n1p15 105M  6.1M   99M   6% /boot/efi
tmpfs           783M  4.0K  783M   1% /run/user/1000
ubuntu@ip-10-0-2-150:~$ sudo docker run -d \
>   --name open-webui \
>   --restart always \
>   -p 3000:8080 \
>   -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \
>   --add-host=host.docker.internal:host-gateway \
>   ghcr.io/open-webui/open-webui:main
Unable to find image 'ghcr.io/open-webui/open-webui:main' locally
main: Pulling from open-webui/open-webui
84a2afebaf4d: Pull complete
4afda7e4a767: Pull complete
4f4fb700ef54: Pull complete
bb97a7af1971: Pull complete
0ca4ca8e967f: Pull complete
71c87fe6927a: Pull complete
0567bba6946b: Pull complete
ade6fdd14ee3: Pull complete
1076a6959a11: Pull complete
4524391b1074: Pull complete
ca10a397ad2d: Pull complete
801abe499289: Pull complete
e221127a7c39: Pull complete
c6aec87f9665: Pull complete
c873187f8cea: Pull complete
e2f8d3915065: Download complete
6671467d2ab5: Download complete
Digest: sha256:c2e4723fdbca5de8f9f0529e22b78acf5bc312a88da730bed88860063d028fe8
Status: Downloaded newer image for ghcr.io/open-webui/open-webui:main
f6ccdaa221b7c042d6077bf83564931a2e698f3bff2a3a9857d4f94f3f8ca1d3
ubuntu@ip-10-0-2-150:~$
```

Figure 31: Docker container (open-webui) running in healthy state, mapped to port 3000:8080, with --restart always ensuring automatic startup on server reboot.

6.6 API Verification via curl

The model responded successfully with a detailed answer about machine learning, confirming the API is fully operational and the fine-tuned model is correctly loaded and serving requests.

```
ubuntu@ip-10-0-2-150:~$ curl http://localhost:11434/api/generate -d '{
>   "model": "llama3.2:3b",
>   "prompt": "What is machine learning?",
>   "stream": false
> }'

{"model":"llama3.2:3b","created_at":"2026-04-27T11:51:39.266439841Z","response":"Machine learning (ML) is a subset of artificial intelligence (AI) that involves training algorithms to learn from data and make p
redictions, decisions, or take actions without being explicitly programmed. The term \"machine learning\" was first coined in the 1950s by computer scientist Arthur Samuel.\n\nIn traditional programming, softwa
re developers write rules-based code to solve problems. In contrast, machine learning allows computers to discover patterns and relationships within large datasets through an iterative process of data analysis,
 model training, and testing.\n\nMachine learning algorithms are designed to learn from experience and improve their performance over time, without being explicitly programmed to perform a specific task. The co
re idea is that the algorithm will analyze the input data, identify relevant features, and develop a predictive model or decision-making strategy based on the patterns it discovers.\n\nThere are several types o
f machine learning:\n\n1. **Supervised Learning**: The algorithm is trained on labeled data, where the correct output is already known. The goal is to learn a mapping between inputs and outputs.\n2. **Unsupervi
sed Learning**: The algorithm is trained on unlabeled data, and the goal is to identify hidden patterns or relationships in the data.\n3. **Reinforcement Learning**: The algorithm learns by interacting with an
environment and receiving feedback in the form of rewards or penalties.\n\nMachine learning has numerous applications in areas such as:\n\n1. **Image and Speech Recognition**\n2. **Natural Language Processing (
NLP)**\n3. **Predictive Maintenance** (e.g., predicting equipment failures)\n4. **Recommendation Systems** (e.g., personalized product recommendations)\n5. **Self-Driving Cars** (e.g., object detection, lane tr
acking)\n\nThe benefits of machine learning include:\n\n1. **Improved Accuracy**: By analyzing large datasets and identifying patterns, algorithms can make more accurate predictions.\n2. **Increased Efficiency*
*: Machine learning can automate many tasks, reducing manual effort and improving productivity.\n3. **Scalability**: Machine learning models can handle vast amounts of data, making them suitable for large-scale
 applications.\n\nHowever, machine learning also has its limitations and challenges, such as:\n\n1. **Data Quality Issues**: Poorly prepared or biased data can lead to inaccurate results.\n2. **Overfitting**: M
odels may become too specialized to the training data and fail to generalize well to new situations.\n3. **Explainability**: It can be difficult to understand how machine learning models make decisions, leading
 to concerns about transparency and accountability.\n\nOverall, machine learning has revolutionized many industries by enabling computers to learn from data and improve their performance over time.","done":true
,"done_reason":"stop","context":[128006,9125,128007,271,38766,1303,33025,2696,25,6790,220,2366,18,271,128009,128006,882,128007,271,3923,374,5780,6975,30,128009,128006,78191,128007,271,22333,6975,320,2735,8,374,
264,27084,315,21075,11478,320,15836,8,430,18065,4967,26249,311,4048,505,828,323,1304,20492,11,11429,11,477,1935,6299,2085,1694,21650,56168,13,578,4751,330,33156,6975,1,574,1176,78718,304,279,220,6280,15,82,555,
6500,28568,28686,33007,382,644,8776,15840,11,3241,13707,3350,5718,6108,2082,311,11886,5435,13,763,13168,11,5780,6975,6276,19002,311,7142,12912,323,12135,2949,3544,30525,1555,459,87975,1920,315,828,6492,11,1646,
4967,11,323,7649,382,22333,6975,26249,527,6319,311,4048,505,3217,323,7417,872,5178,927,892,11,2085,1694,21650,56168,311,2804,264,3230,3465,13,578,6332,4623,374,430,279,12384,690,24564,279,1988,828,11,10765,9959
,4519,11,323,2274,264,60336,1646,477,5597,28846,8446,3196,389,279,12912,433,52114,382,3947,527,3892,4595,315,5780,6975,1473,16,13,3146,10254,79090,21579,96618,578,12384,374,16572,389,30929,828,11,1405,279,4495,
2612,374,2736,3967,13,578,5915,374,311,4048,264,13021,1990,11374,323,16674,627,17,13,3146,1844,13066,79090,21579,96618,578,12384,374,16572,389,65985,23121,828,11,323,279,5915,374,311,10765,8340,12912,477,12135,
304,279,828,627,18,13,3146,697,258,11167,21579,96618,578,12384,47310,555,45830,449,459,4676,323,12588,11302,304,279,1376,315,21845,477,31086,382,22333,6975,706,12387,8522,304,5789,1778,439,1473,16,13,3146,1945,
323,39841,48698,1035,17,13,3146,55381,11688,29225,320,45,12852,8,1035,18,13,3146,54644,535,39306,334,320,68,1326,2637,52997,7241,28950,340,19,13,3146,68744,367,15264,334,320,68,1326,2637,35649,2027,19075,340,20
,13,3146,12363,12,94936,36231,334,320,68,1326,2637,1665,18468,11,21971,15194,696,791,7720,315,5780,6975,2997,1473,16,13,3146,82210,51275,96618,3296,42118,3544,30525,323,25607,12912,11,26249,649,1304,810,13687,2
0492,627,17,13,3146,97941,67667,96618,13257,6975,649,69711,1690,9256,11,18189,11630,5149,323,18899,26206,627,18,13,3146,3407,278,2968,96618,13257,6975,4211,649,3790,13057,15055,315,828,11,3339,1124,14791,369,35
44,13230,8522,382,11458,11,5780,6975,1101,706,1202,9669,323,11774,11,1778,439,1473,16,13,3146,1061,18410,37543,96618,45773,398,10235,477,48761,828,649,3063,311,50504,3135,627,17,13,3146,1959,6410,1303,96618,279
72,1253,3719,2288,28175,311,279,4967,828,323,3775,311,93640,1664,311,502,15082,627,18,13,3146,849,21435,2968,96618,1102,649,387,5107,311,3619,1268,5780,6975,4211,1304,11429,11,6522,311,10742,922,28330,323,39242
,382,28589,11,5780,6975,706,14110,1534,1690,19647,555,28462,19002,311,4048,505,828,323,7417,872,5178,927,892,13],"total_duration":202674399842,"load_duration":262200438,"prompt_eval_count":30,"prompt_eval_durat
ion":350060419,"eval_count":520,"eval_duration":200777527246}ubuntu@ip-10-0-2-150:~$
```

Figure 32: curl API call to Ollama running on EC2 instance (localhost:11434) with the fine-tuned model (**llama3.2:3b** — the local Ollama name assigned to the fine-tuned GGUF model via `ollama create`) responding successfully to the prompt 'What is machine learning?', confirming the model is correctly deployed and serving requests.

Section 7 — Web Interface

7.1 Web Interface Setup

The **web-based chat interface** was deployed using **OpenWebUI**, served via a **Docker container on the EC2 instance** and accessible through a browser at http://44.201.50.135:3000. The interface was configured to start automatically on server restart using **the --restart always Docker flag**, as documented in Section 6.

7.2 Interface Screenshots

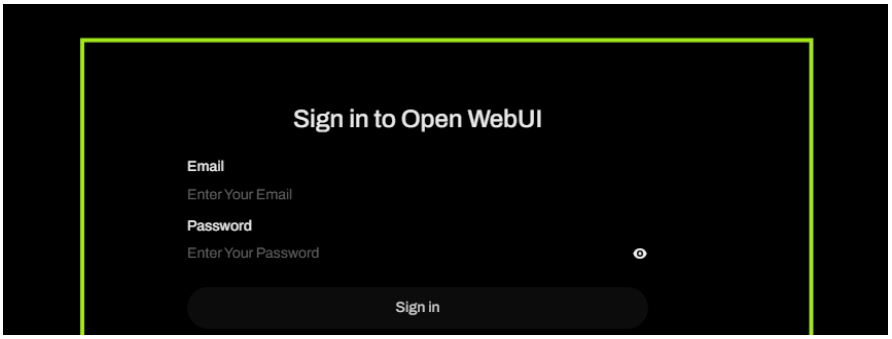


Figure 33: OpenWebUI login page accessible from the browser at the EC2 public IP address on port 3000, confirming the web interface is live and reachable.

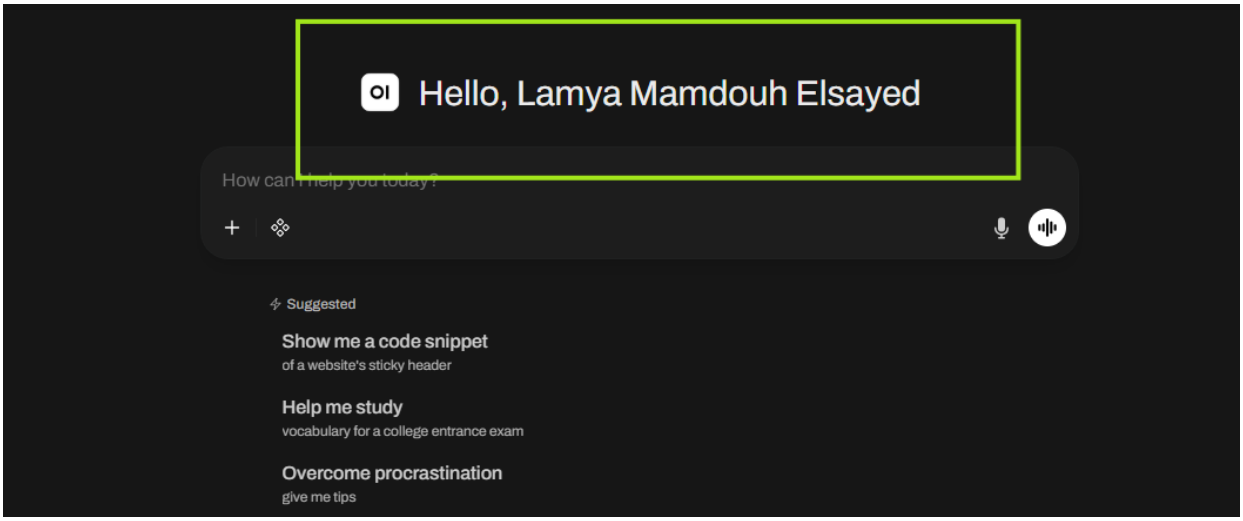


Figure 34: OpenWebUI main interface after login, showing the chat assistant ready to receive user input with the greeting "Hello, Lamya Mamdouh Elsayed", confirming successful authentication and interface functionality.

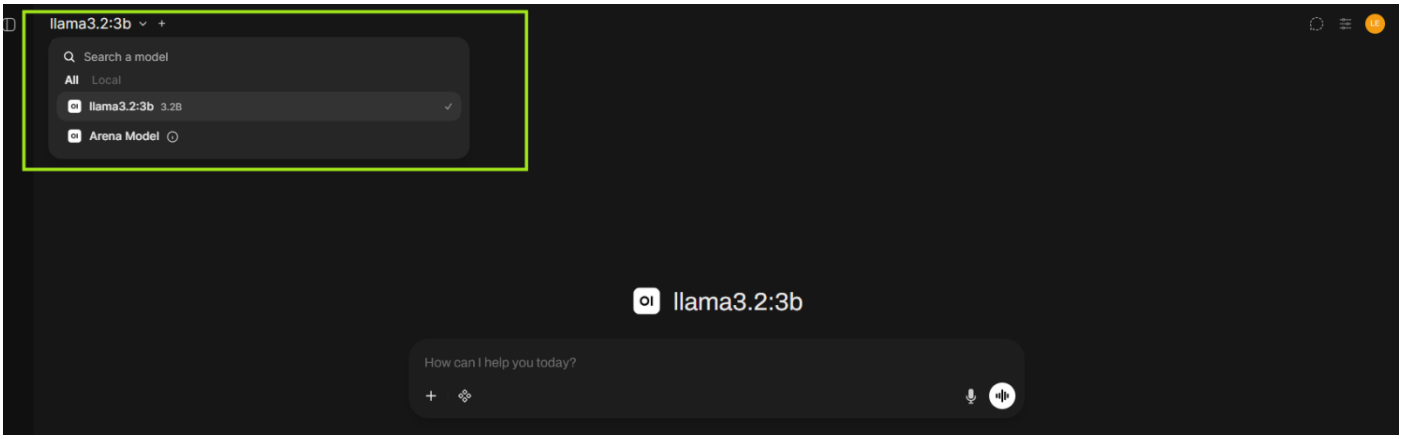


Figure 35: OpenWebUI model selector showing the fine-tuned model llama3.2:3b (3.2B) selected and active as the local model, confirming the fine-tuned model is correctly loaded and visible in the interface.

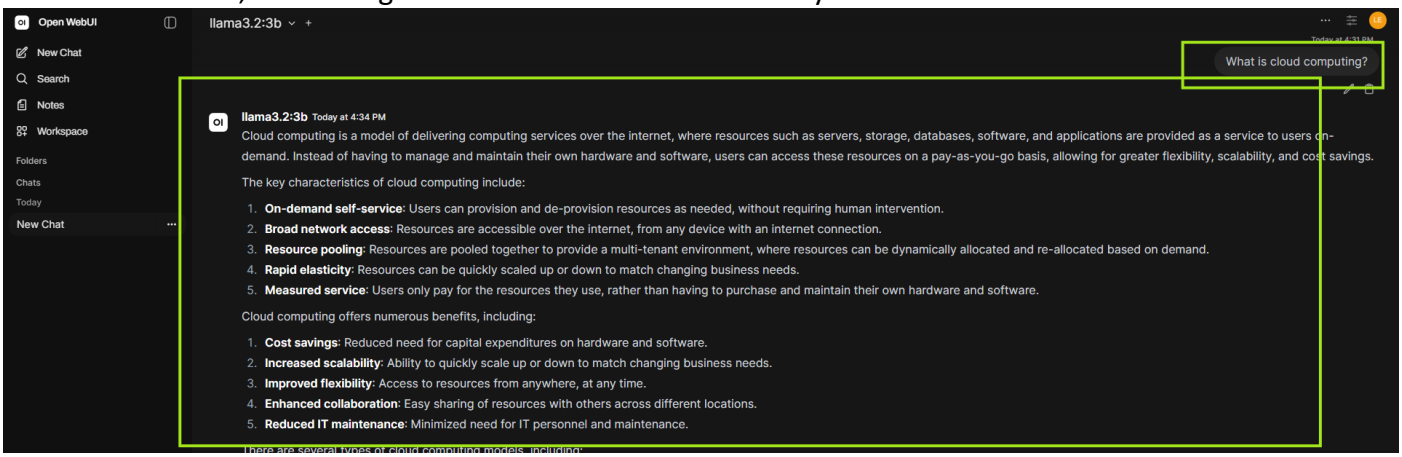


Figure 36: Sample conversation with the deployed model through **OpenWebUI.** The user asked **"What is cloud computing?"** and the fine-tuned **model (llama3.2:3b)** responded with a detailed, structured answer covering the definition, key characteristics, benefits, and types of cloud computing. The browser URL bar shows the **EC2 public IP address (44.201.50.135:3000),** confirming live deployment.